

Operating System (OS): Process ID (PID)

1. What is Process ID (PID)?

Definition:

A **Process ID (PID)** is a **unique identification number** assigned by the Operating System to every running process. It allows the OS to uniquely identify, manage, and control each process.

Example

Process Name	Process ID (PID)
chrome.exe	2548
notepad.exe	7352
explorer.exe	2456

At any given time, no two running processes have the same PID. Once a process is terminated, its PID may be reused by the operating system for a new process.

Why is PID Important?

- **Process Identification:** Identifies each running process uniquely.
- **Process Management:** Allows the OS to monitor and manage processes.
- **Process Termination:** Enables users or the OS to terminate a specific process using its PID.
- **Parent-Child Relationship:** Maintains relationships between parent and child processes.

2. What is Process Suspension?

Definition:

Process Suspension is the process of **temporarily stopping (pausing) the execution of a process without terminating it**. The suspended process can later continue execution from the same point.

Process State Transition

A process does not become suspended instantly. It goes through the following transition:

Running
↓
Suspending
↓
Suspended

Suspend vs. Terminate

Suspend	Terminate
Temporarily stops the process.	Permanently ends the process.
The process can be resumed later.	The process cannot be resumed.
The process state is preserved.	The process state is destroyed.
CPU usage becomes 0% while suspended.	All allocated resources are released.

3. What Happens Internally When a Process is Suspended?

When the operating system suspends a process, three major operations take place.

A. State Change

The process state changes to either:

- **Suspended Ready**
 - **Suspended Blocked**
-

B. Resource and CPU Release

The operating system removes the process from CPU execution.

- CPU usage becomes **0%**.
 - CPU time becomes available for other processes.
-

C. Context Saving

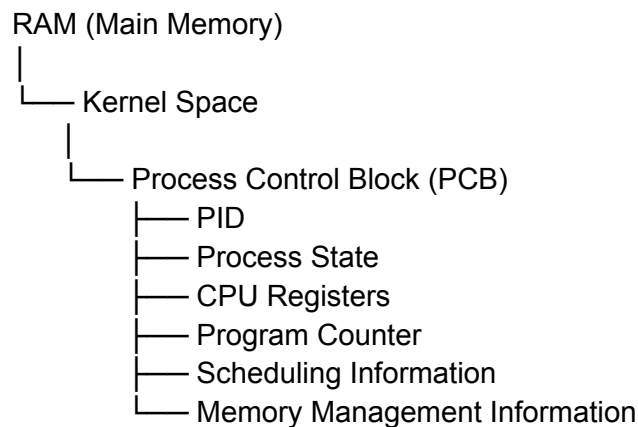
The operating system saves the complete execution context of the process, including:

- CPU Registers
- Program Counter (PC)
- Stack Pointer
- Stack
- Other processor state information

This allows the process to continue later from exactly where it stopped.

4. Where is the Process State Stored?

All important information about a process is stored in a special operating system data structure called the **Process Control Block (PCB)**.



Memory Location Details

PCB Location

The **PCB** is stored in the **Kernel Space of RAM**.

Even if a process is suspended, its PCB normally remains in RAM so that the operating system can manage and resume the process quickly.

Memory Swapping (Swap Out)

If the system experiences **low memory**, the operating system may move the process's memory image (such as code, data, stack, and heap) from RAM to secondary storage.

- **Linux:** Swap Space
- **Windows:** Pagefile

This operation is called **Swap Out**.

Important Note:

- **PCB remains in RAM.**
 - **Only the process memory (code, data, stack, heap) may be moved to disk when necessary.**
-

5. Why Does the Operating System Suspend Processes?

The operating system suspends processes for several reasons.

Memory Management

When RAM becomes insufficient, less important background processes may be suspended to free memory.

Power Saving

Suspended processes consume almost no CPU time, helping reduce power consumption and extend battery life.

Example:

Background applications on Windows and smartphones are often suspended automatically.

Deadlock Recovery

If a deadlock or resource conflict occurs, the operating system may suspend one or more processes temporarily to recover system resources and maintain system stability.

6. Practical Commands and Usage

Linux / Unix

Suspend a Process

Press:

Ctrl + Z

This sends the **SIGSTOP** signal to suspend the running process.

Resume a Process

Use:

fg

This sends the **SIGCONT** signal and resumes the suspended process in the foreground.

Windows

Task Manager

You can suspend or resume certain processes through:

Task Manager



Performance



Open Resource Monitor



Right-click Process

↓
Suspend Process / Resume Process

Sleep Mode

Sleep Mode is a practical example of process suspension.

When the computer enters Sleep Mode:

- Running processes are suspended.
 - Their execution state remains stored in RAM.
 - The computer enters a low-power state.
 - When the computer wakes up, the processes resume from where they were suspended.
-

Exam Definition (Short)

Process ID (PID)

A Process ID (PID) is a unique identification number assigned by the operating system to every running process for identification and management.

Process Suspension

Process Suspension is the temporary stopping of a process without terminating it, while preserving its execution state so that it can be resumed later.

Yesterday 11:11 PM

ভালো প্রশ্ন। এগুলো দুইটি state-এর মধ্যে ভাগ করে বুঝতে হবে।

Embryo (New/Starting) State-এ যা হয়

- ☒ The process has already been created.
- ☒ A PID has been assigned.
- ☒ The PCB (Process Control Block) has been created.
- ☒ Memory has been allocated to the process.

-  Required resources (memory, files, etc.) are allocated.

এরপর Process Ready Queue-তে পাঠানো হয়।

Ready State-এ যা হয়

-  The process is placed in the Ready Queue.
 -  The process is waiting for the CPU Scheduler to assign the CPU.
 -  CPU ব্যবহার করছে না।
 -  কোনো instruction execute হচ্ছে না।
 -  CPU পেলেই Running State-এ চলে যাবে।
-

সহজে মনে রাখো

Embryo/New State = Process তৈরি করার কাজ

- PID তৈরি
- PCB তৈরি
- Memory Allocate
- Resource Allocate

Ready State = Process প্রস্তুত, শুধু CPU-এর অপেক্ষা করছে

- Ready Queue-তে থাকে
- Scheduler-এর জন্য অপেক্ষা করে
- CPU পেলেই Running হবে

এটাই পরীক্ষার জন্য সবচেয়ে সঠিকভাবে দুই state-এর পার্থক্য।

Running state bolo

Running State

Definition

Running State is the state in which a process is currently executing instructions using the CPU.

What happens in the Running State?

-  The process is assigned the CPU by the CPU Scheduler.
-  The CPU executes the process instructions.
-  The Program Counter (PC) continuously updates to point to the next instruction.

- ✓ CPU register values are continuously updated.
 - ✓ The process may access memory, files, and I/O devices.
 - ✓ The process performs computations and executes its assigned tasks.
-

CPU Status

- ✓ The process is using the CPU.
 - ✓ Instructions are being executed.
 - ✗ The process is not waiting for the CPU.
-

Possible Transitions from Running State

1. Running → Ready

- The process's time quantum (time slice) expires.
- The CPU Scheduler removes it from the CPU (descheduling/preemption).

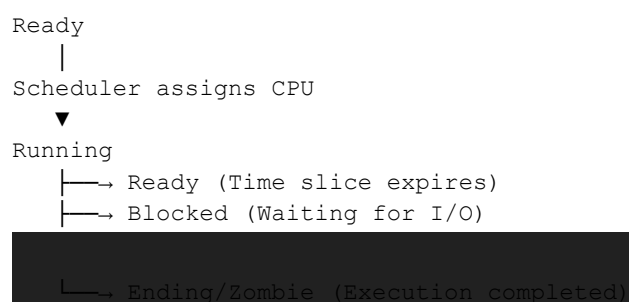
2. Running → Blocked

- The process requests an I/O operation (e.g., keyboard input, disk read, network communication).
- It waits until the I/O operation is completed.

3. Running → Ending (Zombie)

- The process completes its execution.
 - It enters the Ending/Zombie state before being removed from the system.
-

Simple Flow



Exam Definition

Running State is the state in which a process is actively executing instructions using the CPU.

zombie state...

Zombie State (Ending State)

Definition

Zombie State is the state in which a process has finished its execution, but its Process Control Block (PCB) and exit status are still kept by the operating system until the parent process collects them.

What happens in the Zombie State?

-  The process has completed its execution.
 -  The process does not use the CPU.
 -  The process does not execute any instructions.
 -  Most of the process memory and other resources are released.
 -  Only the PCB and exit status remain in the kernel.
 -  The operating system waits for the parent process to collect the child's exit status using `wait()` or `waitpid()`.
-

CPU Status

-  CPU is not used.
 -  No instructions are executed.
-

Memory Status

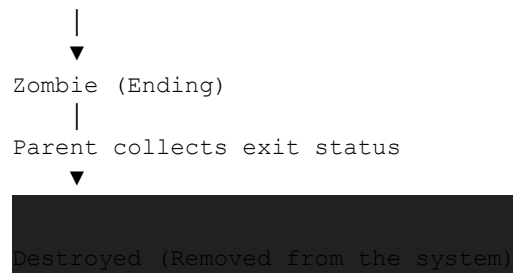
-  Most memory (code, data, stack, heap) is released.
 -  Only the PCB and exit information remain in kernel memory (RAM).
-

Why does a Zombie Process exist?

A zombie process exists because the parent process has not yet collected the child's termination status. After the parent process collects this information, the operating system completely removes the process and deletes its PCB.

Transition

Running



Exam Definition

Zombie State is the state in which a process has completed execution, but its PCB and exit status remain in the operating system until the parent process collects them.

Process Context Switching – Definition

Process Context Switching is the process by which the operating system saves the context (state) of the currently running process and restores the context of another process, allowing the CPU to switch execution from one process to another.

Simple Definition (Exam)

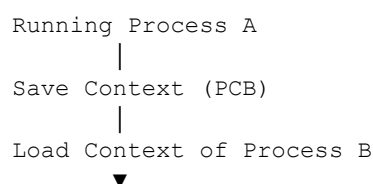
Context Switching is the process of saving the state of the current process and loading the state of the next process so that the CPU can switch from one process to another.

What is saved during Context Switching?

- Program Counter (PC)
- CPU Registers
- Stack Pointer (SP)
- Process State
- Other CPU-related information

These are saved in the Process Control Block (PCB).

State Transition



One-line exam definition:

Context Switching is the process of saving the execution context of one process and restoring the execution context of another process to enable CPU sharing.

context switching beshi hole kii hobe and kom hole ki hobe ?

Context Switching যত বেশি বা কম হবে, তার সরাসরি প্রভাব CPU Performance-এর উপর পড়ে।

1. Context Switching বেশি হলে

কী হয়?

- ❌ CPU-এর অনেক সময় Context Save ও Context Restore করতে নষ্ট হয়।
- ❌ কম সময় Process-এর আসল কাজ (Execution) হয়।
- ❌ CPU Overhead বেড়ে যায়।
- ❌ System Performance কমে যায়।
- ❌ Throughput কমে যায় (কম সংখ্যক process সম্পন্ন হয়)।
- ❌ Response Time কিছু ক্ষেত্রে বেড়ে যেতে পারে।

উদাহরণ

ধরো,

- একটি Context Switch = 5 μ s
- ১ সেকেন্ডে 100,000 বার Context Switch হচ্ছে।

তাহলে CPU-এর অনেক সময় শুধু Process বদলাতেই চলে যাবে, কাজ করার সময় কমে যাবে।

2. Context Switching কম হলে

কী হয়?

- ✅ CPU বেশি সময় Process Execute করতে পারে।
- ✅ CPU Overhead কম হয়।
- ✅ System Performance ভালো হয়।
- ✅ Throughput বৃদ্ধি পায়।
- ✅ CPU Efficiency বাড়ে।

কিন্তু...

যদি Context Switching অত্যন্ত কম হয়,

তাহলে—

- একটি Process অনেকক্ষণ CPU দখল করে রাখতে পারে।
- অন্য Ready Process-গুলোকে বেশি অপেক্ষা করতে হয়।
- Response Time খারাপ হতে পারে।
- Interactive System-এ User Experience খারাপ হতে পারে।

Comparison

More Context Switching	Less Context Switching
High CPU overhead	Low CPU overhead
Lower CPU efficiency	Higher CPU efficiency
Lower system performance	Better system performance
Less execution time for processes	More execution time for processes
Too frequent process switching	Longer CPU usage by each process

Exam Answer (2 Marks)

More Context Switching

- Increases CPU overhead.
- Reduces system performance.
- Decreases CPU efficiency.

Less Context Switching

- Reduces CPU overhead.
- Improves CPU efficiency.
- Increases overall system performance.

One-line conclusion

Frequent context switching wastes CPU time due to excessive overhead, while an appropriate amount of context switching improves overall system performance and responsiveness.

context switch er theke ki ki content ache sob context golo amk bolo book theke ekdom

যদি তোমার Operating System বইতে Context Switching অধ্যায় থাকে, তাহলে সাধারণত নিচের টপিকগুলোই থাকে। এগুলো প্রায় সব স্ট্যান্ডার্ড OS বই (Silberschatz, Galvin, Stallings, Tanenbaum, xv6 lecture notes) এবং বিশ্ববিদ্যালয়ের কোর্সে পড়ানো হয়।

Context Switching Topics

1. Definition of Context Switching

- What is Context Switching?
- Why is Context Switching needed?

2. Process Context

- What is Process Context?
- Components of Process Context

3. Context (What is Saved?)

Context Switching-এর সময় কী কী Save হয়?

- Program Counter (PC)
- CPU Registers
- Stack Pointer (SP)
- Processor Status Word (PSW/Flags)
- Memory Management Information
- Process State
- Scheduling Information

4. Process Control Block (PCB)

- Definition
- Structure

- Why Context is saved in PCB
-

5. Steps of Context Switching

Typical sequence:

1. Interrupt/System Call
 2. Save Current Process Context
 3. Update PCB
 4. Scheduler Runs
 5. Select Next Process
 6. Load New Process Context
 7. Resume Execution
-

6. Scheduler

- Long-Term Scheduler
 - Short-Term Scheduler
 - Medium-Term Scheduler
-

7. Dispatcher

- Definition
 - Functions
 - Dispatcher vs Scheduler
 - Dispatch Latency
-

8. Descheduling

- Definition
 - Running → Ready
 - Running → Blocked
-

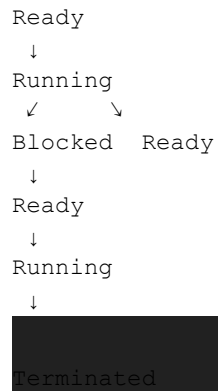
9. Scheduling Queue

- Job Queue
 - Ready Queue
 - Device Queue
-

10. Process State Transition

New

↓



11. Causes of Context Switching

- Time Quantum Expired
- Higher Priority Process
- Interrupt
- I/O Request
- System Call
- Process Termination

12. Types of Context Switching

- Process Context Switch
- Thread Context Switch

13. Context Switch Overhead

- Saving Registers
- Loading Registers
- Cache Effects
- TLB Flush (in many systems)

14. Advantages

- Multitasking
- Better CPU Utilization
- Fair Scheduling
- Improved Responsiveness

15. Disadvantages

- CPU Overhead

- Performance Loss
 - Cache Miss
 - Increased Latency
-

16. Context Switch Time

- Definition
 - Factors Affecting Context Switch Time
-

17. Context Switching vs Process Switching

Difference between

- Context Switch
- Process Switch

18. Context Switching vs Interrupt

Comparison

19. Context Switching vs Multitasking

Difference

পরীক্ষার জন্য সবচেয়ে গুরুত্বপূর্ণ প্রশ্ন

1. Definition of Context Switching
2. Why Context Switching is needed?
3. What information is saved during Context Switching?
4. Explain the steps of Context Switching.
5. Context Switching using PCB.
6. Advantages and Disadvantages of Context Switching.
7. Scheduler and Dispatcher.
8. Context Switching vs Process Switching.
9. Context Switch Overhead.
10. Causes of Context Switching.

এই ১০টি টপিক থেকেই সাধারণত বিশ্ববিদ্যালয়ের Operating System পরীক্ষায় Context Switching সম্পর্কিত প্রায় সব প্রশ্ন আসে।

1. Ultimate Operating System Exam Notes
2.  **Chapter List**
3. **Chapter 1 : Introduction to Process**
4. Process
5. Program vs Process
6. Process Control Block (PCB)
7. Process ID (PID)

Chapter 2 : Process States

5. Embryo (New) State
6. Ready State
7. Scheduler
8. Running State
9. Descheduler
10. Blocked (Waiting) State
11. Zombie (Ending) State
12. Complete Process State Diagram

Chapter 3 : Process Suspension

13. Process Suspension
14. Suspend vs Terminate
15. Suspended Ready
16. Suspended Blocked
17. Swap Space
18. Page File
19. Sleep Mode

Chapter 4 : Context Switching

20. Process Context
21. Context Switching
22. Why Context Switching?
23. Steps of Context Switching
24. Information Saved
25. PCB and Context Switching

- 26. Context Switching Overhead
- 27. Advantages
- 28. Disadvantages
- 29. More vs Less Context Switching
- 30. Causes
- 31. Context Switch Time

8. Chapter 8 : Exam Questions

- 9. 1 Mark Questions
- 10. 2 Mark Questions
- 11. 5 Mark Questions
- 12. Viva Questions
- 13. MCQ Points
- 14. Important Definitions
- 15. Diagrams
- 16. Short Notes

Preemptive Scheduling

Definition

Preemptive Scheduling is a CPU scheduling method in which the Operating System can interrupt a running process and allocate the CPU to another process before the current process completes its execution.

When is it Used?

- When a higher-priority process arrives.
- In **time-sharing systems**.
- In **interactive systems**, where quick response is required.
- When the CPU time quantum expires (e.g., in **Round Robin Scheduling**).

Example

P1 is currently using the CPU.

Suddenly, **P2** arrives with a higher priority.

The Operating System will:

1. Save the context of **P1**.
2. Allocate the CPU to **P2**.
3. After **P2** finishes or releases the CPU, **P1** resumes execution.

Advantages

- Fast response time.
- Better CPU utilization.
- High-priority processes are executed quickly.
- Supports time-sharing systems.

Disadvantages

- More context switching occurs.
- Higher CPU overhead.
- More complex scheduler implementation.

Examples of Preemptive Scheduling Algorithms

- Round Robin (RR)
 - Shortest Remaining Time First (SRTF)
 - Preemptive Priority Scheduling
-

Non-Preemptive Scheduling

Definition

Non-Preemptive Scheduling is a CPU scheduling method in which a process keeps the CPU until it finishes its execution or requests an I/O operation. The Operating System cannot take the CPU away from a running process.

When is it Used?

- In **batch processing systems**.
- Where response time is not critical.
- In simple operating systems.
- When minimizing context switching is important.

Example

Suppose:

- **P1** is using the CPU.
- Then **P2** arrives.

In this case:

- **P2** must wait.
- **P1** continues running until it finishes or blocks for I/O.
- Only then does **P2** get the CPU.

Advantages

- Less context switching.
- Lower CPU overhead.
- Simple implementation.
- Less CPU time is wasted.

Disadvantages

- High-priority processes may have to wait.
- Longer response time.
- Long-running processes may delay other processes (**Convoy Effect**).

Examples of Non-Preemptive Scheduling Algorithms

- First Come First Served (FCFS)
 - Non-Preemptive Priority Scheduling
 - Non-Preemptive Shortest Job First (SJF)
-

Preemptive vs Non-Preemptive

Feature	Preemptive	Non-Preemptive
CPU can be taken away from a running process	 Yes	 No
Context Switching	More	Less
Response Time	Faster	Slower

Complexity	Higher	Lower
Best For	Interactive, Real-Time, Time-Sharing Systems	Batch Processing Systems
Examples	RR, SRTF, Preemptive Priority	FCFS, SJF, Non-Preemptive Priority

Easy Way to Remember

- **Preemptive** = The Operating System can interrupt a running process and take the CPU away.
- **Non-Preemptive** = A process keeps the CPU until it finishes execution or voluntarily releases it.

17.

 Ultimate Operating System Exam Notes - Complete Chapter by Chapter

18.

19.---

20.

21.##  CHAPTER 1: INTRODUCTION TO PROCESS

22.

23.---

24.

25.### 1. What is a Process?

26.

27.**Definition.**

28. A process is a program in execution. It is an active entity that includes the program code, current activity, stack, data section, and process control block.

29.

30. **Program vs Process:**

31. | Program | Process |

32. |-----|-----|

33. | Passive entity | Active entity |

34. | Stored on disk | Loaded in memory |

35. | Static | Dynamic |

36. | Contains only code | Contains code, data, stack, PCB |

37. | No execution state | Has execution state |

38.

39. **Process Components:**

40. 1. **Text Section** - Program code

41. 2. **Data Section** - Global variables

42. 3. **Heap** - Dynamically allocated memory

43. 4. **Stack** - Temporary data (function parameters, return addresses)

44. 5. **Program Counter (PC)** - Next instruction address

45.

46. ---

47.

48. **2. Process ID (PID)**

49.

50. **Definition:**

51. A Process ID (PID) is a unique identification number assigned by the Operating System to every running process for identification and management.

52.

53. **Example:**

54. | Process Name | Process ID (PID) |

55. |-----|-----|

56. | chrome.exe | 2548 |

57. | notepad.exe | 7352 |

58. | explorer.exe | 2456 |

59.

60. **Importance:**

61.  **Process Identification** - Uniquely identifies each process

62.  **Process Management** - Allows OS to monitor and manage processes

63.  **Process Termination** - Enables killing specific processes using PID

64.  **Parent-Child Relationship** - Maintains relationships between processes

65.

66. **Key Points:**

67. - No two running processes have the same PID

68. - Once terminated, PID may be reused

69.- PID is stored in the Process Control Block (PCB)

70.

71.---

72.

73.### 3. Process Control Block (PCB)

74.

75.**Definition:**

76.A Process Control Block (PCB) is a data structure maintained by the Operating System that contains all important information about a process.

77.

78.**Components of PCB:**

79.``

80.			
81.	PROCESS CONTROL BLOCK		
82.			
83.	1. Process ID (PID)		
84.	2. Process State		
85.	3. Program Counter (PC)		
86.	4. CPU Registers		
87.	5. Stack Pointer		
88.	6. Memory Management Info		
89.	7. Scheduling Information		
90.	8. I/O Status Information		
91.	9. List of Open Files		
92.	10. Accounting Information		
93.			

94.``

95.

96.**Location:**

97.``

98.RAM (Main Memory)

99.

100.└─ Kernel Space

101.

102.└─ Process Control Block (PCB)

103.``

104.

105.**Why PCB is Important:**

106.- Contains all process information

107.- Allows context switching

108.- Enables process management

109.- Stored in kernel memory

110.

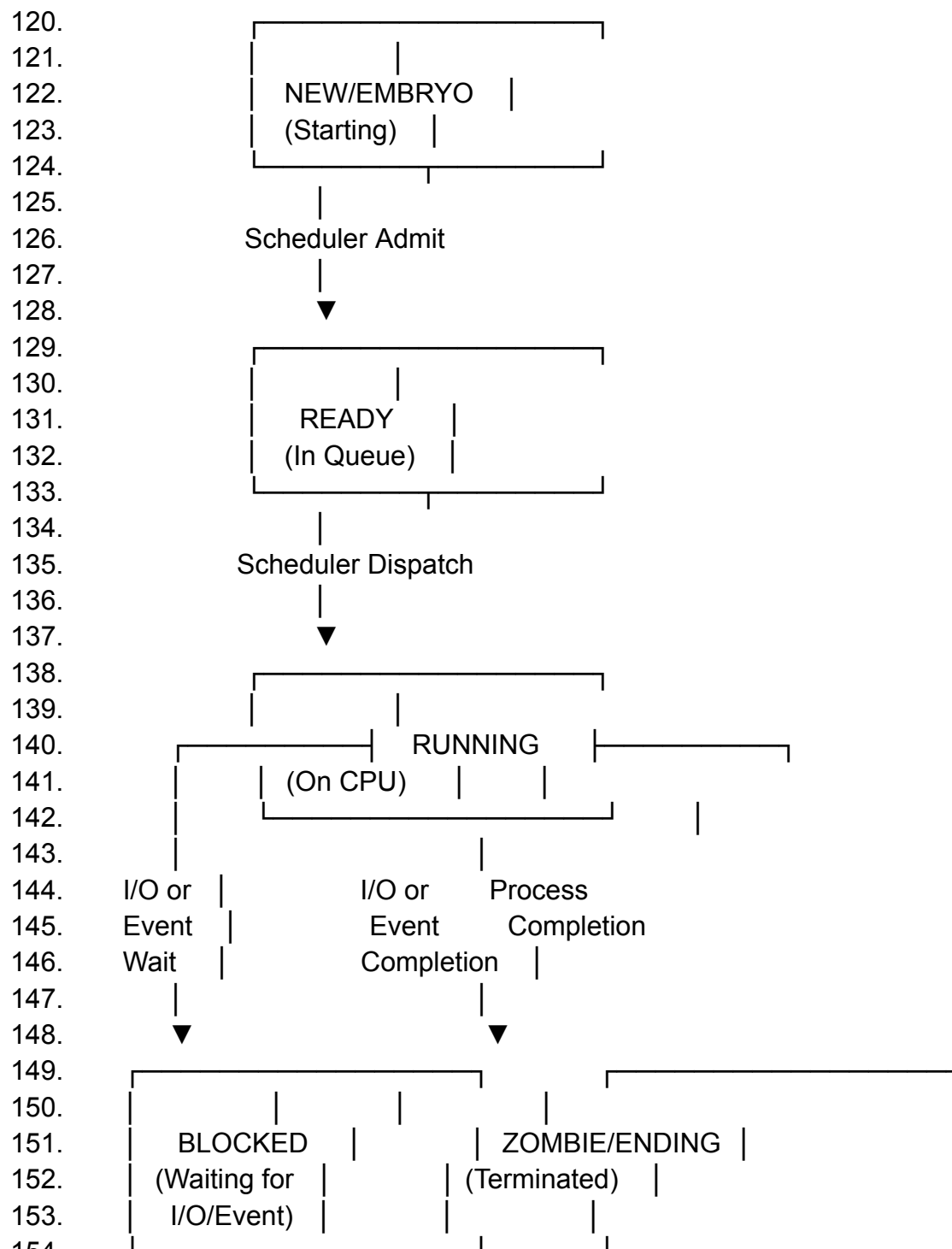
111.---

112.
113. ## 📖 CHAPTER 2: PROCESS STATES

114.
115. ---

116.
117. ### 4. Process State Diagram

118.
119. ...



154.
155. ...

156.

157. ---

158.

159. ### 5. Embryo (New) State

160.

161. **Definition:**

162. The Embryo State is the initial state of a process when it is being created by the Operating System.

163.

164. **What happens in Embryo State:**

165. ✓ The process has already been created

166. ✓ A PID has been assigned

167. ✓ The PCB has been created

168. ✓ Memory has been allocated to the process

169. ✓ Required resources are allocated

170. ✓ Process is ready to enter Ready Queue

171.

172. **Process Creation Steps:**

173. ```

174. 1. Process Creation Request

175. |

176. ▼

177. 2. PID Assigned

178. |

179. ▼

180. 3. PCB Created

181. |

182. ▼

183. 4. Memory Allocated

184. |

185. ▼

186. 5. Resources Allocated

187. |

188. ▼

189. 6. Ready Queue Entry

190. ```

191.

192. **Simple Memory:**

193. Embryo/New State = **Process Creation Phase**

194. - PID তৈরি

195. - PCB তৈরি

196. - Memory Allocate

197. - Resource Allocate

198.

199. ---

200.

201. ### 6. Ready State

202.

203. **Definition:**

204. The Ready State is the state in which a process is fully prepared to execute but is waiting for the CPU Scheduler to assign the CPU.

205.

206. **What happens in Ready State:**

207. ☒ Process is placed in the **Ready Queue**

208. ☒ Process is waiting for the **CPU Scheduler**

209. ☒ All necessary resources are allocated

210. ☐ Process is **NOT using the CPU**

211. ☐ No instructions are being executed

212. ☒ Process will move to Running State when CPU is assigned

213.

214. **Transitions:**

215. ```

216. Embryo → Ready (When created)

217. Running → Ready (When time quantum expires)

218. Blocked → Ready (When I/O completes)

219. ```

220.

221. ---

222.

223. ### 7. Scheduler

224.

225. **Definition:**

226. A Scheduler is an Operating System component that selects which process from the Ready Queue will be assigned to the CPU.

227.

228. **Types of Schedulers:**

229. | Scheduler Type | Purpose | Frequency |

230. |-----|-----|-----|

231. | **Long-Term Scheduler** | Selects processes to be loaded into memory | Rarely |

232. | **Short-Term Scheduler** | Selects process from Ready Queue for CPU | Very frequently |

233. | **Medium-Term Scheduler** | Removes/swaps processes from memory | Moderate |

234.

235. **Short-Term Scheduler:**

236. ```

237. Ready Queue

238. |
 239. ▼
 240. Short-Term Scheduler
 241. |
 242. ▼
 243. Selects Next Process
 244. |
 245. ▼
 246. CPU Assigned (Running)

247. ...

248.

249. ---

250.

251. ### 8. Running State

252.

253. **Definition:**

254. Running State is the state in which a process is actively executing instructions using the CPU.

255.

256. **What happens in Running State:**

257. ✓ Process is assigned the CPU by Scheduler

258. ✓ CPU executes process instructions

259. ✓ Program Counter (PC) continuously updates

260. ✓ CPU register values are continuously updated

261. ✓ Process may access memory, files, and I/O devices

262. ✓ Process performs computations and executes tasks

263.

264. **CPU Status:**

265. ✓ Process is using the CPU

266. ✓ Instructions are being executed

267. ✗ Process is NOT waiting for CPU

268.

269. **Transitions from Running State:**

270. ...

271. Running

272. | → Ready (Time slice expires - Preemption)

273. | → Blocked (I/O Request)

274. | → Zombie (Process completes execution)

275. ...

276.

277. ---

278.

279. ### 9. Descheduler

280.

281. ****Definition:****

282. A Descheduler is a component that removes a running process from the CPU and places it back into the Ready Queue or Blocked State.

283.

284. ****When Descheduling Happens:****

285. 1. ****Time Quantum Expires**** - Running → Ready

286. 2. ****Higher Priority Process**** - Running → Ready (Preemption)

287. 3. ****I/O Request**** - Running → Blocked

288. 4. ****Process Completion**** - Running → Zombie

289.

290. ****Descheduling Process:****

291. ```

292. Running Process

293. |
294. ▼

295. Save Context to PCB

296. |
297. ▼

298. Update Process State

299. |
300. ▼

301. Move to Appropriate Queue

302. |
303. ▼

304. Scheduler Selects Next Process

305. ```

306.

307. ---

308.

309. **### 10. Blocked (Waiting) State**

310.

311. ****Definition:****

312. The Blocked State is the state in which a process is waiting for some event (I/O completion, resource availability, etc.) and cannot continue execution.

313.

314. ****What happens in Blocked State:****

315. ☒ Process is waiting for an event

316. ☒ CPU is not used

317. ☒ No instructions are executed

318. ☒ Process remains in memory

319. ☒ Will move to Ready when event completes

320.

321. ****Common Causes of Blocking:****

322. 1. ****I/O Operations**** - Reading from disk, keyboard, network

323. 2. ****Resource Unavailability**** - Waiting for mutex, semaphore
 324. 3. ****Event Waiting**** - Waiting for timer, signal
 325.
 326. ****Transition:****
 327. ```
 328. Running → Blocked (I/O Request)
 329. Blocked → Ready (I/O Complete)
 330. ```
 331.
 332. ---
 333.
 334. **### 11. Zombie (Ending) State**
 335.
 336. ****Definition:****
 337. Zombie State is the state in which a process has completed its execution,
 but its Process Control Block (PCB) and exit status remain in the Operating
 System until the parent process collects them.
 338.
 339. ****What happens in Zombie State:****
 340. ☒ Process has completed execution
 341. ☒ Process does NOT use the CPU
 342. ☒ No instructions are executed
 343. ☒ Most memory (code, data, stack, heap) is released
 344. ☒ Only PCB and exit status remain
 345. ☒ OS waits for parent to collect exit status
 346.
 347. ****Memory Status:****
 348. ```
 349. Memory Status in Zombie:
 350. |—— Code Section → Released
 351. |—— Data Section → Released
 352. |—— Stack → Released
 353. |—— Heap → Released
 354. |—— PCB → Remains in Kernel
 355. ```
 356.
 357. ****Why Zombie Exists:****
 358. - Parent process hasn't called `wait()` or `waitpid()`
 359. - After parent collects info, process is fully destroyed
 360.
 361. ****Transition:****
 362. ```
 363. Running
 364. |

365. ▼
366. Zombie (Ending)
367. |
368. Parent collects exit status
369. |
370. ▼
371. Destroyed (Completely Removed)
372. ```
373.
374. **Exam Definition:**
375. Zombie State is the state in which a process has completed execution, but its PCB and exit status remain in the Operating System until the parent process collects them.
376.
377. ---
378.
379. ## 📖 CHAPTER 3: PROCESS SUSPENSION
380.
381. ---
382.
383. ### 12. Process Suspension
384.
385. **Definition:**
386. Process Suspension is the temporary stopping of a process without terminating it, while preserving its execution state so that it can be resumed later.
387.
388. **Process State Transition:**
389. ```
390. Running
391. ↓
392. Suspending
393. ↓
394. Suspended
395. ```
396.
397. **Internal Operations during Suspension:**
398.
399. **A. State Change:**
400. The process state changes to either:
401. - Suspended Ready
402. - Suspended Blocked
403.
404. **B. Resource and CPU Release:**

405. - Process removed from CPU execution
 406. - CPU usage becomes 0%
 407. - CPU time becomes available for other processes
 408.
 409. ****C. Context Saving:****
 410. The OS saves the complete execution context:
 411. - CPU Registers
 412. - Program Counter (PC)
 413. - Stack Pointer
 414. - Stack
 415. - Other processor state information
 416.
 417. ---
 418.
 419. **### 13. Suspend vs Terminate**
 420.
 421. | Feature | Suspend | Terminate |
 422. |-----|-----|-----|
 423. | ****Duration**** | Temporarily stops | Permanently ends |
 424. | ****Resume**** | Can be resumed later | Cannot be resumed |
 425. | ****State**** | Preserved | Destroyed |
 426. | ****CPU Usage**** | Becomes 0% | Completely released |
 427. | ****Resources**** | Held (or swapped) | All released |
 428. | ****PCB**** | Retained | Destroyed |
 429.
 430. ---
 431.
 432. **### 14. Suspended Ready vs Suspended Blocked**
 433.
 434. ****Suspended Ready:****
 435. - Process was in ****Ready State****
 436. - Suspended by OS
 437. - Not ready to execute
 438. - In memory or swapped to disk
 439. - Can be resumed → Ready
 440.
 441. ****Suspended Blocked:****
 442. - Process was in ****Blocked State****
 443. - Suspended by OS
 444. - Waiting for I/O
 445. - May be swapped to disk
 446. - Event completion → Suspended Ready
 447.
 448. ****Diagram:****

449. ```
450. Running → Suspending → Suspended Ready
451. Running → Suspending → Suspended Blocked
452.
453. Suspended Ready → Ready (After Resume)
454. Suspended Blocked → Suspended Ready (After I/O Complete)
455. ```
456.
457. ---

458.
459. ### 15. Swap Space

460.
461. **Definition:**
462. Swap Space is a dedicated area on secondary storage (disk) where the
Operating System moves processes when RAM is insufficient.

463.
464. **Memory Swapping:**
465. ```
466. RAM (Main Memory)
467. |
468. |— Kernel Space (PCB stays here)
469. |
470. |— User Space (Process Memory)
471. |
472. | (When RAM is full)
473. ▼
474. Secondary Storage
475. |
476. |— Swap Space (Linux)
477. |
478. |— Pagefile (Windows)
479. ```

480.
481. **Key Points:**
482. - PCB remains in RAM
483. - Only process memory (code, data, stack, heap) may be moved
484. - Swapping helps manage memory efficiently

485.
486. **Why Swap Happens:**
487. 1. Memory is low
488. 2. Less important processes
489. 3. Background processes
490. 4. Power saving
491.

492. ---
493.
494. ### 16. Sleep Mode
495.
496. **Definition:**
497. Sleep Mode is a practical example of process suspension where all processes are suspended, and the computer enters a low-power state.
498.
499. **What happens in Sleep Mode:**
500. - Running processes are suspended
501. - Execution state remains in RAM
502. - Computer enters low-power state
503. - When woken, processes resume from where they stopped
504.
505. **Memory Management:**
506. ```
507. Normal Operation:
508. RAM (Active) ← All processes
509.
510. Sleep Mode:
511. RAM (Low Power) ← Process states preserved
512.
513. Wake Up:
514. RAM (Active) ← Process states restored
515. ```
516.
517. ---
518.
519. ## 📖 CHAPTER 4: CONTEXT SWITCHING
520.
521. ---
522.
523. ### 17. Context Switching - Definition
524.
525. **Definition:**
526. Context Switching is the process of saving the execution context of one process and restoring the execution context of another process to enable CPU sharing.
527.
528. **One-Line Definition:**
529. Context Switching is the process of saving the state of the current process and loading the state of the next process so that the CPU can switch from one process to another.
530.

531. ****Context Switch Visualization:****

532. ```

533. Process A Running

534. |

535. ▼

536. Save Context of A → PCB of A

537. |

538. ▼

539. Load Context of B ← PCB of B

540. |

541. ▼

542. Process B Running

543. ```

544.

545. ---

546.

547. **### 18. What is Saved During Context Switching?**

548.

549. ****Information Saved in PCB:****

550. ```

551.		
552.	CONTEXT SAVED IN PCB	
553.		
554.	1. Program Counter (PC)	
555.	2. CPU Registers (General Purpose)	
556.	3. Stack Pointer (SP)	
557.	4. Process State	
558.	5. Processor Status Word (PSW/Flags)	
559.	6. Memory Management Information	
560.	7. Scheduling Information	
561.	8. I/O Status Information	
562.		

563. ```

564.

565. ****Why Save All This?****

566. - Allows process to continue from the exact same point

567. - Preserves all CPU-related information

568. - Enables seamless switching between processes

569.

570. ---

571.

572. **### 19. Steps of Context Switching**

573.

574. ****Complete Context Switching Process:****

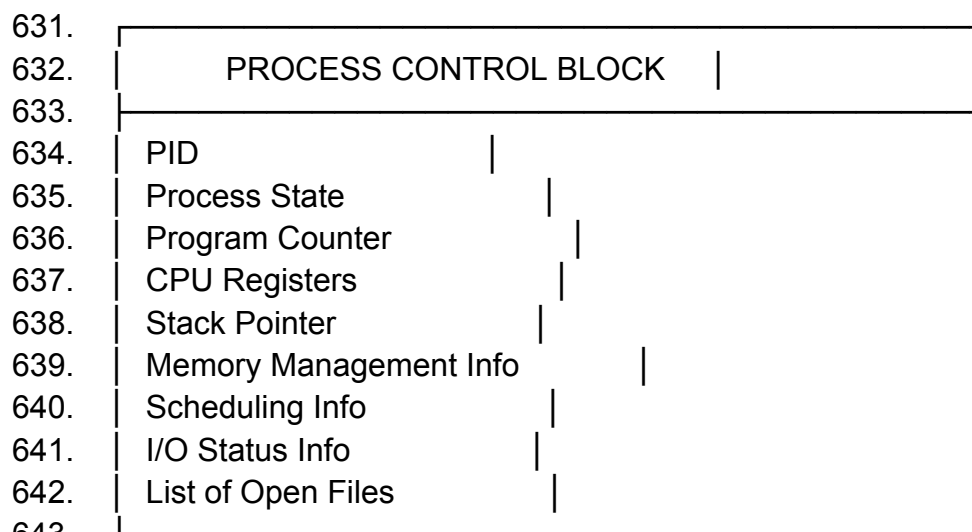
```

575.
576.   ``
577. Step 1: Interrupt or System Call Occurs
578.   |
579.   ▼
580. Step 2: Save Context of Current Process
581.   - Save PC to PCB
582.   - Save Registers to PCB
583.   - Save Stack Pointer to PCB
584.   - Save Process State
585.   |
586.   ▼
587. Step 3: Update PCB of Current Process
588.   - Change State: Running → Ready/Blocked
589.   - Update Scheduling Information
590.   |
591.   ▼
592. Step 4: Move Process to Appropriate Queue
593.   - Ready Queue OR Blocked Queue
594.   |
595.   ▼
596. Step 5: Short-Term Scheduler Runs
597.   - Select Next Process from Ready Queue
598.   |
599.   ▼
600. Step 6: Load Context of New Process
601.   - Load PC from PCB
602.   - Load Registers from PCB
603.   - Load Stack Pointer from PCB
604.   |
605.   ▼
606. Step 7: Resume Execution of New Process
607.   ``
608.
609. **Simple Flow:**
610.   ``
611. Save Context → Update PCB → Select Next Process → Load Context →
    Resume
612.   ``
613.
614. ---
615.
616. ### 20. Context Switching Components
617.

```

618. ****A. Context (What is Saved?)****
 619. *******
 620. Program Counter (PC) - Next instruction address
 621. CPU Registers - General purpose registers
 622. Stack Pointer (SP) - Top of stack
 623. Processor Status Word (PSW) - Flags and status
 624. Memory Management Info - Page table base, TLB
 625. Scheduling Info - Priority, time quantum
 626. Process State - Current state (Ready, Running, etc.)
 627. *******

628.
 629. ****B. Process Control Block (PCB)****
 630. *******



643. *******
 644. *******
 645.
 646. ****C. Scheduler****
 647. *******
 648. Types:
 649. 1. Long-Term Scheduler - Loads processes into memory
 650. 2. Short-Term Scheduler - Selects process for CPU
 651. 3. Medium-Term Scheduler - Swaps processes in/out
 652. *******

653.
 654. ****D. Dispatcher****
 655. *******
 656. Functions:
 657. 1. Context Switching
 658. 2. Switching to User Mode
 659. 3. Jumping to Proper Location
 660. *******
 661.

662. ****Dispatcher vs Scheduler:****
663. | Scheduler | Dispatcher |
664. |-----|-----|
665. | Selects which process to run | Performs the actual switch |
666. | Decision making | Action execution |
667. | Runs less frequently | Runs frequently |
668. | Part of scheduling algorithm | Low-level mechanism |
669.
670. ****Dispatch Latency:****
671. - Time taken by dispatcher to stop one process and start another
672. - Should be as low as possible
673.
674. ---
675.
676. **### 21. Causes of Context Switching**
677.
678. ****1. Time Quantum Expires****
679. - Process has used its CPU time slice
680. - Preemptive scheduling
681.
682. ****2. Higher Priority Process Arrives****
683. - New process with higher priority
684. - Preemptive scheduling
685.
686. ****3. I/O Request****
687. - Process needs I/O operation
688. - Can't continue without it
689.
690. ****4. Interrupt****
691. - Hardware or software interrupt
692. - CPU needs to handle it
693.
694. ****5. System Call****
695. - Process requests OS service
696. - May require context switch
697.
698. ****6. Process Termination****
699. - Process completes execution
700. - OS needs to schedule next
701.
702. ****7. Voluntary Yield****
703. - Process voluntarily releases CPU
704. - Non-preemptive scheduling
705.

706. ---
707.
708. ### 22. Context Switching Overhead
709.
710. **Definition:**
711. Context Switching Overhead refers to the time and resources consumed
by the OS when performing a context switch.
712.
713. **Components of Overhead:**
714.
715. ```
716.

CONTEXT SWITCH OVERHEAD	
1. Saving Registers	
2. Loading Registers	
3. Cache Effects (Cache Miss)	
4. TLB Flush (on many systems)	
5. Scheduler Execution Time	
6. Dispatcher Execution Time	

726. ```
727.
728. **Factors Affecting Overhead:**
729. 1. Number of registers to save/restore
730. 2. Processor speed
731. 3. Cache size
732. 4. TLB implementation
733. 5. Scheduling algorithm complexity
734. 6. Operating system design
735.
736. **Overhead Example:**
737. ```
738. If Context Switch Time = 5 μ s
739. Process Time Quantum = 100 ms
740. Overhead = $5/100000 = 0.005\%$ (very small)
741.
742. But if Context Switch = 5 μ s
743. Time Quantum = 1 ms
744. Overhead = $5/1000 = 0.5\%$ (significant)
745. ```
746.
747. ---
748.

749. ### 23. More vs Less Context Switching

750.

751. **More Context Switching:**

752. | Effect | Description |

753. |-----|-----|

754. |  High CPU Overhead | Much time wasted saving/restoring contexts |

755. |  Lower CPU Efficiency | Less time for actual execution |

756. |  Poor System Performance | Throughput decreases |

757. |  Less Execution Time | Each process gets less CPU time |

758. |  Better Responsiveness | Interactive systems respond faster |

759. |  Faster Response Time | Quicker reaction to user input |

760.

761. **Less Context Switching:**

762. | Effect | Description |

763. |-----|-----|

764. |  Low CPU Overhead | More time for execution |

765. |  Higher CPU Efficiency | Better throughput |

766. |  Better System Performance | More processes completed |

767. |  More Execution Time | Each process gets more CPU time |

768. |  Poor Responsiveness | Processes may starve |

769. |  Higher Response Time | Users may experience delays |

770.

771. **Comparison Table:**

772. | Feature | More Context Switching | Less Context Switching |

773. |-----|-----|-----|

774. | CPU Overhead | High | Low |

775. | CPU Efficiency | Low | High |

776. | System Performance | Lower | Better |

777. | Response Time | Better | Worse |

778. | Process Starvation | Less likely | More likely |

779. | Throughput | Lower | Higher |

780.

781. **Exam Conclusion:**

782. > Frequent context switching wastes CPU time due to excessive overhead, while an appropriate amount of context switching improves overall system performance and responsiveness.

783.

784. ---

785.

786. ### 24. Advantages and Disadvantages of Context Switching

787.

788. **Advantages:**

789. ``

790. |

791.	ADVANTAGES OF CONTEXT SWITCHING	
792.		
793.	✓ Enables Multitasking	
794.	- Multiple processes share CPU	
795.		
796.	✓ Better CPU Utilization	
797.	- CPU never idle for long	
798.		
799.	✓ Fair Scheduling	
800.	- All processes get CPU time	
801.		
802.	✓ Improved Responsiveness	
803.	- Interactive systems work well	
804.		
805.	✓ Supports Time-Sharing	
806.	- Multiple users can use system	
807.		
808.	✓ High Priority Process Handling	
809.	- Important tasks get priority	
810.		

811. ```

812.

813. **Disadvantages:**

814. ```

815.	DISADVANTAGES OF CONTEXT SWITCHING	
816.		
817.		
818.	✗ CPU Overhead	
819.	- Time wasted on switching	
820.		
821.	✗ Cache Misses	
822.	- Cache gets flushed/invalidated	
823.		
824.	✗ TLB Flush (in many systems)	
825.	- Address translation cache cleared	
826.		
827.	✗ Performance Loss	
828.	- Overall system performance drops	
829.		
830.	✗ Increased Latency	
831.	- Switching takes time	
832.		
833.	✗ Complex Implementation	
834.	- OS needs sophisticated scheduler	

835. _____

836. ```

837.

838. ---

839.

840. ### 25. Context Switch Time

841.

842. **Definition:**

843. Context Switch Time is the time required for the Operating System to save the context of one process and load the context of another process.

844.

845. **Context Switch Time Calculation:**

846. ```

847. Context Switch Time = Save Time + Load Time + Scheduler Time

848.

849. Where:

850. - Save Time = Time to save registers, PC, SP, etc.

851. - Load Time = Time to load registers, PC, SP, etc.

852. - Scheduler Time = Time for scheduler to select next process

853. ```

854.

855. **Factors Affecting Context Switch Time:**

856.

857. ```

858. 1. Number of Registers

859. └─ More registers → More time

860.

861. 2. CPU Speed

862. └─ Faster CPU → Less time

863.

864. 3. Memory Speed

865. └─ Faster RAM → Less time

866.

867. 4. Cache Size

868. └─ Larger cache → Less time for TLB

869.

870. 5. TLB Implementation

871. └─ Hardware TLB → Less time

872.

873. 6. OS Implementation

874. └─ Efficient OS → Less time

875.

876. 7. Process Size

877. └─ Larger process → More time (if memory info saved)

878. ```

879.

880. ****Typical Values:****

881. ```

882. Simple System: 1-5 microseconds

883. Complex System: 10-50 microseconds

884. Worst Case: Up to 100+ microseconds

885. ```

886.

887. ****Impact on System:****

888. ```

889. If Context Switch = 10 μ s

890. Time Quantum = 100 ms

891. Overhead = 0.01%

892.

893. If Context Switch = 10 μ s

894. Time Quantum = 1 ms

895. Overhead = 1%

896. ```

897.

898. ---

899.

900. **## 📖 CHAPTER 5: SCHEDULING (PREEMPTIVE vs NON-PREEMPTIVE)**

901.

902. ---

903.

904. **### 26. Preemptive Scheduling**

905.

906. ****Definition:****

907. Preemptive Scheduling is a CPU scheduling method in which the Operating System can interrupt a running process and allocate the CPU to another process before the current process completes its execution.

908.

909. ****When Preemption Occurs:****

910. ```

911. 1. Higher Priority Process Arrives

912. └─ OS interrupts current process

913.

914. 2. CPU Time Quantum Expires

915. └─ Round Robin Scheduling

916.

917. 3. System Call or Interrupt

918. └─ OS takes control

919.
920. 4. I/O Request
921. └─ Process voluntarily gives up CPU
922. ...
923.
924. **Example:**
925. ...
926. P1 is using CPU (Time: 0-10 ms)
927. P2 arrives (Higher Priority) at 5 ms
928. |
929. ▼
930. OS Saves Context of P1
931. |
932. ▼
933. OS Loads Context of P2
934. |
935. ▼
936. P2 Runs (Time: 5-15 ms)
937. |
938. ▼
939. P2 Finishes or Releases CPU
940. |
941. ▼
942. P1 Resumes from where it stopped
943. ...
944.
945. **Advantages:**
946. ...
947. ✓ Fast Response Time
948. └─ Interactive users get quick feedback
949.
950. ✓ Better CPU Utilization
951. └─ CPU is always busy
952.
953. ✓ High Priority Processes Execute Quickly
954. └─ Critical tasks get CPU immediately
955.
956. ✓ Supports Time-Sharing Systems
957. └─ Multiple users share CPU fairly
958. ...
959.
960. **Disadvantages:**
961. ...
962. ✗ More Context Switching

963. └─ Higher CPU overhead
 964.
 965. ✗ Higher CPU Overhead
 966. └─ Time wasted on switching
 967.
 968. ✗ Complex Scheduler Implementation
 969. └─ OS code is more complicated
 970. ...
 971.
 972. **Preemptive Scheduling Algorithms:**
 973. ...
 974. 1. Round Robin (RR)
 975. └─ Each process gets fixed time quantum
 976.
 977. 2. Shortest Remaining Time First (SRTF)
 978. └─ Process with shortest remaining time gets CPU
 979.
 980. 3. Preemptive Priority Scheduling
 981. └─ Higher priority process preempts lower priority
 982. ...
 983.
 984. ---
 985.
 986. ### 27. Non-Preemptive Scheduling
 987.
 988. **Definition:**
 989. Non-Preemptive Scheduling is a CPU scheduling method in which a
 process keeps the CPU until it finishes its execution or requests an I/O
 operation. The Operating System cannot take the CPU away from a running
 process.
 990.
 991. **When Process Releases CPU:**
 992. ...
 993. 1. Process Completes Execution
 994. └─ Naturally terminates
 995.
 996. 2. Process Requests I/O
 997. └─ Voluntarily gives up CPU
 998.
 999. 3. Process Calls yield()
 1000. └─ Voluntarily releases CPU
 1001. ...
 1002.
 1003. **Example:**

1004. ...
1005. P1 is using CPU (Time: 0-20 ms)
1006. P2 arrives at 5 ms
1007. P3 arrives at 10 ms
1008. |
1009. ▼
1010. P1 continues running
1011. |
1012. ▼
1013. P1 finishes at 20 ms
1014. |
1015. ▼
1016. P2 gets CPU (Time: 20-40 ms)
1017. ...
1018.
1019. **Advantages:**
1020. ...
1021. ✓ Less Context Switching
1022. └─ Lower CPU overhead
1023.
1024. ✓ Lower CPU Overhead
1025. └─ More time for execution
1026.
1027. ✓ Simple Implementation
1028. └─ Scheduler is easy to design
1029.
1030. ✓ Less CPU Time Wasted
1031. └─ No unnecessary switches
1032. ...
1033.
1034. **Disadvantages:**
1035. ...
1036. ✗ High Priority Processes May Wait
1037. └─ Critical tasks get delayed
1038.
1039. ✗ Longer Response Time
1040. └─ Interactive systems suffer
1041.
1042. ✗ Convoy Effect
1043. └─ Long processes delay short ones
1044.
1045. ✗ Process Starvation Possible
1046. └─ Some processes may wait indefinitely
1047. ...

1048.

1049. ****Non-Preemptive Scheduling Algorithms:****

1050. ```

1051. 1. First Come First Served (FCFS)

1052. └─ Process that arrives first gets CPU

1053.

1054. 2. Non-Preemptive Priority Scheduling

1055. └─ Higher priority process gets CPU

1056.

1057. 3. Shortest Job First (SJF)

1058. └─ Shortest process gets CPU first

1059. ```

1060.

1061. ---

1062.

1063. **### 28. Preemptive vs Non-Preemptive - Complete Comparison**

1064.

Feature	Preemptive	Non-Preemptive
CPU Interruption	✓ Can be taken away	✗ Cannot be taken
Context Switching	More frequent	Less frequent
CPU Overhead	Higher	Lower
Response Time	Faster	Slower
Complexity	Higher	Lower
CPU Utilization	Better	Lower
Throughput	Variable	Generally higher
Fairness	More fair	Less fair
Best For	Interactive, Real-Time	Batch Processing
Process Starvation	Less likely	More likely
Convoy Effect	Less common	More common
Examples	RR, SRTF, Priority	FCFS, SJF, Non-Preemptive Priority

1079.

1080. ****Easy Way to Remember:****

1081. ```

1082. Preemptive = OS Can Interrupt Running Process

1083. Non-Preemptive = Process Keeps CPU Until Done

1084.

1085. Preemptive = "I can take CPU from you"

1086. Non-Preemptive = "You keep CPU until you're done"

1087. ```

1088.

1089. ---

1090.

1091. **## 📖 CHAPTER 6: SCHEDULING QUEUES**

1092.
1093. ---
1094.
1095. ### 29. Process Scheduling Queues

1096.
1097. **Definition:**
1098. Process Scheduling Queues are data structures maintained by the
Operating System to hold processes in different states.

1099.
1100. **Queue Types:**

1101.
1102. ```

1103.

JOB QUEUE	
(All processes in the system)	

1107.

↓

1109.

READY QUEUE	
(Processes in Ready State)	
- Waiting for CPU	

1114.

↓

1116.

DEVICE QUEUE	
(Processes waiting for I/O)	
- Device 1 Queue	
- Device 2 Queue	
- Device N Queue	

1123. ```

1124.
1125. **Queue Components:**

1126.
1127. ```

1128. 1. Job Queue
- 1129. |— All processes in the system
 - 1130. |— Includes new, ready, running, blocked
 - 1131. |— Maintained in secondary storage
- 1132.
1133. 2. Ready Queue
- 1134. |— Processes in Ready State

- 1135. |— Waiting for CPU allocation
- 1136. |— Implemented as linked list or array
- 1137.
- 1138. 3. Device Queue (I/O Queue)
- 1139. |— Processes waiting for I/O device
- 1140. |— One queue per device
- 1141. |— Example: Disk Queue, Printer Queue

1142. ```

1143.

1144. ****Queue Operations:****

1145. ```

1146. Enqueue: Add process to queue

1147. Dequeue: Remove process from queue

1148. Peek: View next process without removing

1149. ```

1150.

1151. ---

1152.

1153. **## 📖 CHAPTER 7: KEY DIAGRAMS AND TABLES**

1154.

1155. ---

1156.

1157. **### 30. Complete Process State Diagram (With Suspension)**

1158.

1159. ```

1160.

1161.

1162.

1163.

1164.

1165.

1166.

1167.

1168.

1169.

1170.

1171.

1172.

1173.

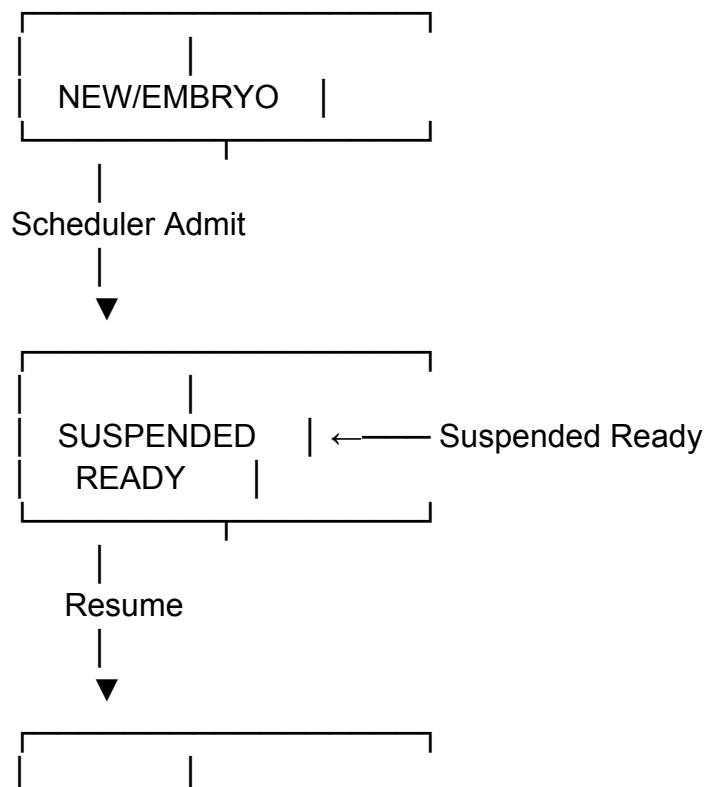
1174.

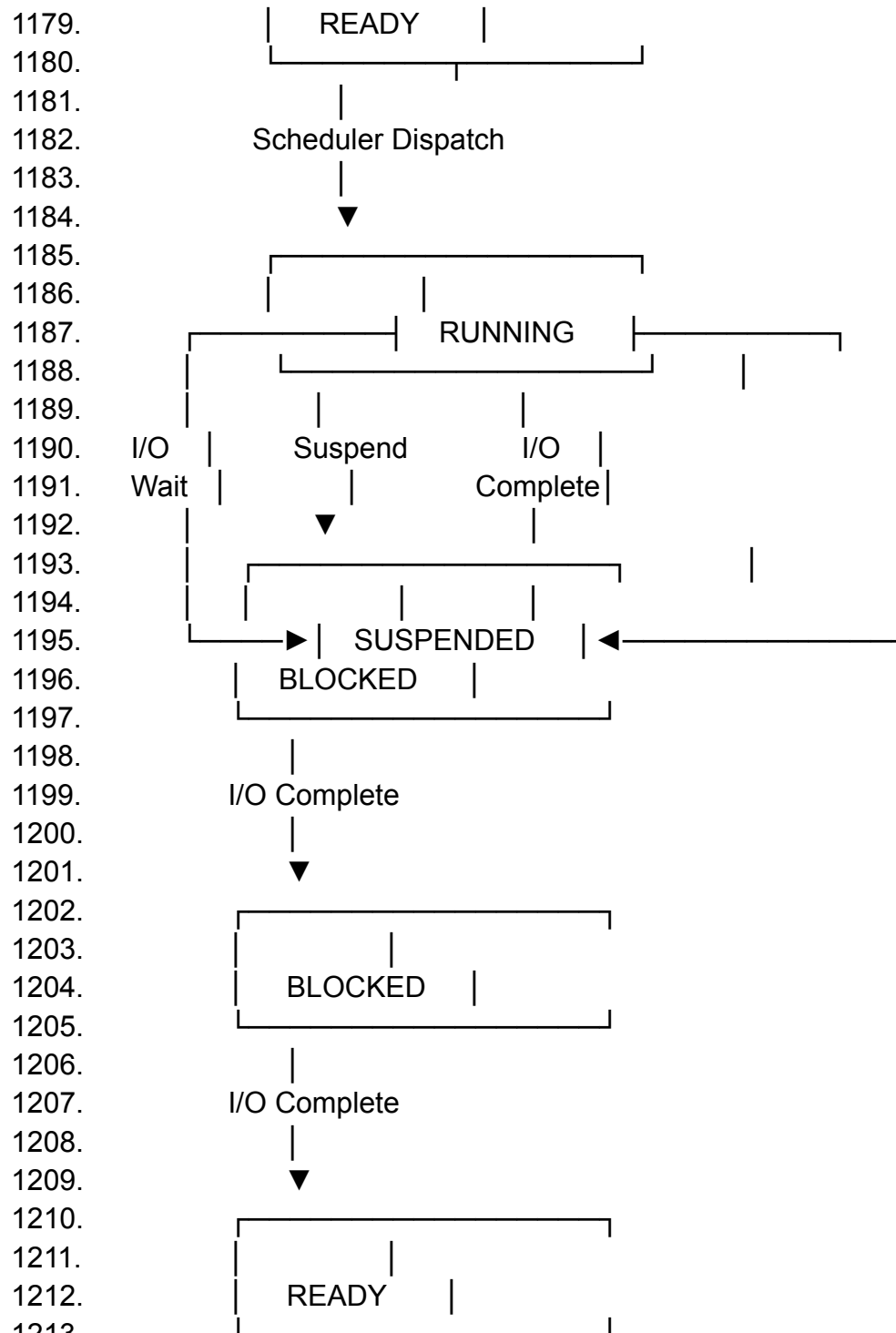
1175.

1176.

1177.

1178.

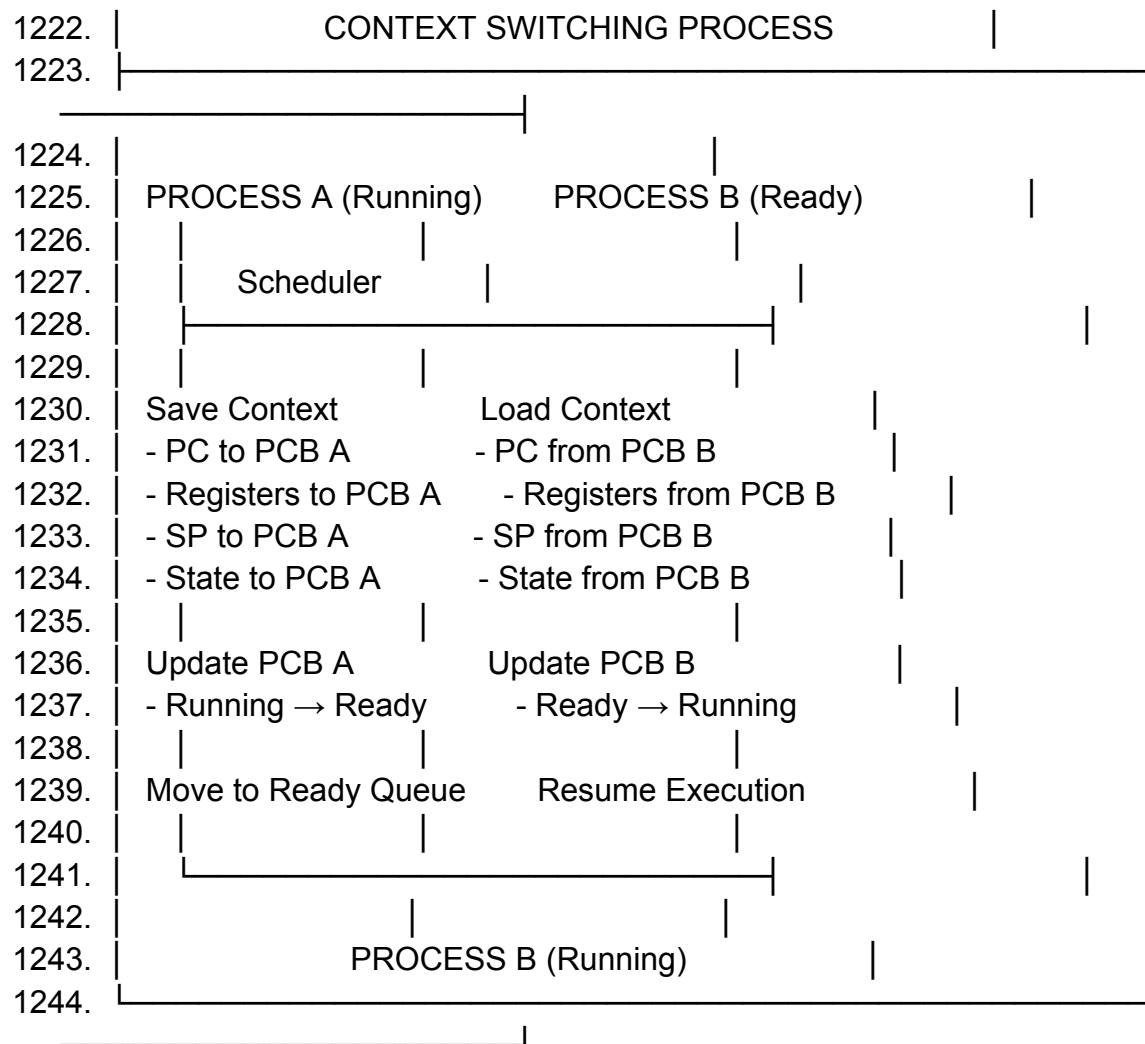




1218. ### 31. Context Switching Complete Diagram

1220. ...

1221. _____



1245. ```

1246.

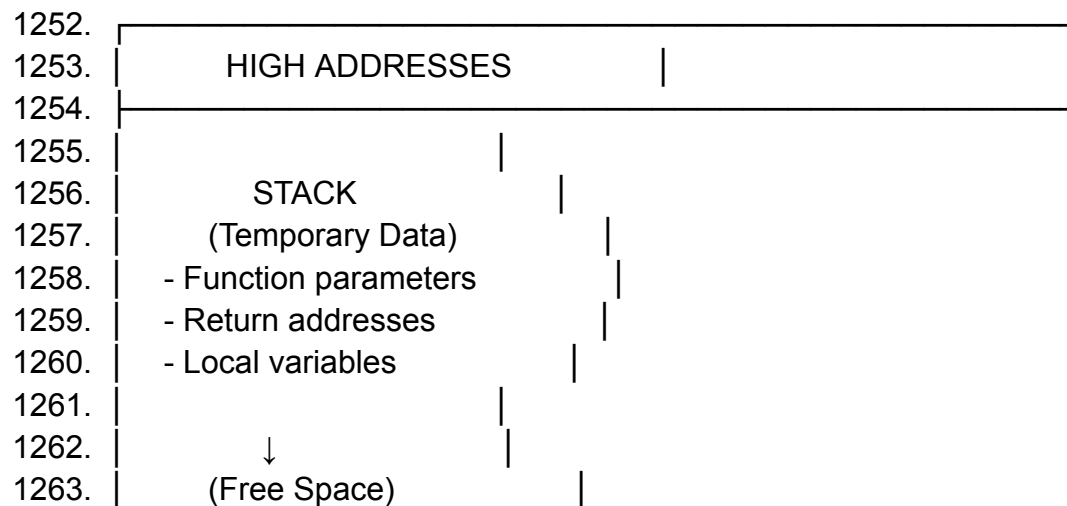
1247. ---

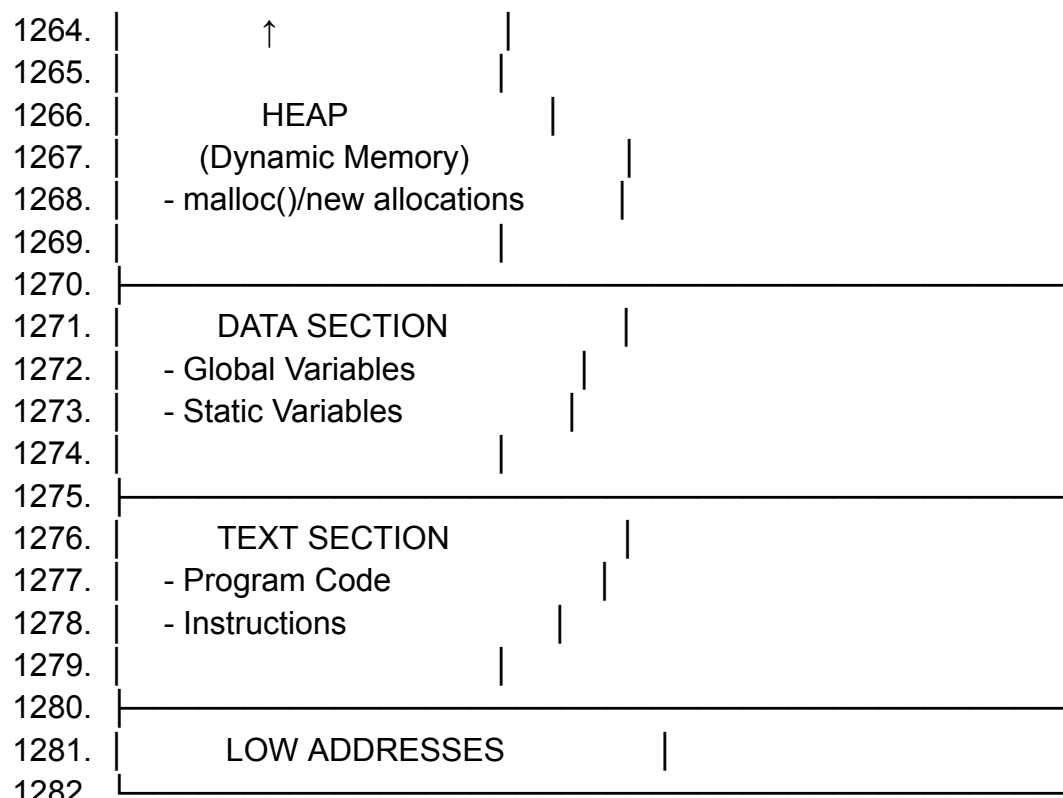
1248.

1249. ### 32. Memory Layout of a Process

1250.

1251. ```





1283. ```
 1284.
 1285. ---
 1286.
 1287. ### 33. Scheduling Comparison Table

1288.				
1289.				
1290.				
1291.	PREEMPTIVE vs NON-PREEMPTIVE SCHEDULING			
1292.				
1293.				
1294.	Criteria	Preemptive	Non-Preemptive	
1295.				
1296.				
1297.	CPU Interruption	Can be interrupted	Cannot be interrupted	
1298.				
1299.	Context Switch	More frequent	Less frequent	
1300.				
1301.	CPU Overhead	Higher	Lower	
1302.				
1303.	Response Time	Faster	Slower	

1304.				
1305.	Complexity	Higher	Lower	
1306.				
1307.	CPU Utilization	Better	Lower	
1308.				
1309.	Throughput	Variable	Generally higher	
1310.				
1311.	Fairness	More fair	Less fair	
1312.				
1313.	Starvation	Less likely	More likely	
1314.				
1315.	Convoy Effect	Less common	More common	
1316.				
1317.	Best For	Interactive	Batch Processing	
1318.		Real-Time		
1319.		Time-Sharing		
1320.				
1321.	Examples	Round Robin	FCFS	
1322.		SRTF	SJF	
1323.		Priority (Preemp.)	Priority (Non-Preemp.)	
1324.				
1325.				

1326. ``

1327.

1328. ---

1329.

1330. ## 📖 CHAPTER 8: EXAM QUESTIONS AND ANSWERS

1331.

1332. ---

1333.

1334. ### 34. 1 Mark Questions with Answers

1335.

1336. **Q1: What is a Process ID (PID)?**

1337. > A PID is a unique identification number assigned by the OS to every running process.

1338.

1339. **Q2: Define Context Switching.**

1340. > Context Switching is the process of saving the context of one process and loading the context of another.

1341.

1342. **Q3: What is PCB?**

1343. > PCB (Process Control Block) is a data structure containing all information about a process.

- 1344.
1345. **Q4: What is the Zombie State?**
1346. > Zombie State is when a process has completed execution but its PCB remains until parent collects exit status.
- 1347.
1348. **Q5: What is Preemptive Scheduling?**
1349. > Preemptive Scheduling allows the OS to interrupt a running process and assign CPU to another.
- 1350.
1351. **Q6: What is Non-Preemptive Scheduling?**
1352. > Non-Preemptive Scheduling is where a process keeps the CPU until it finishes or requests I/O.
- 1353.
1354. **Q7: What is Process Suspension?**
1355. > Process Suspension is temporarily stopping a process without terminating it.
- 1356.
1357. **Q8: What is a Ready Queue?**
1358. > Ready Queue contains processes in Ready State waiting for CPU allocation.
- 1359.
1360. **Q9: What is a Program Counter?**
1361. > Program Counter holds the address of the next instruction to be executed.
- 1362.
1363. **Q10: What is Swap Space?**
1364. > Swap Space is disk area where OS moves process memory when RAM is insufficient.
- 1365.
1366. ---
- 1367.
1368. ### 35. 2 Mark Questions with Answers
- 1369.
1370. **Q1: Explain the difference between Program and Process.**
1371. ``
1372. Program (Passive):
1373. - Stored on disk
1374. - Contains only code
1375. - Static entity
- 1376.
1377. Process (Active):
1378. - Loaded in memory
1379. - Contains code, data, stack, PCB
1380. - Dynamic entity

1381. ```
1382.
1383. **Q2: What information is stored in PCB?**
1384. ```
1385. 1. Process ID (PID)
1386. 2. Process State
1387. 3. Program Counter (PC)
1388. 4. CPU Registers
1389. 5. Stack Pointer (SP)
1390. 6. Memory Management Info
1391. 7. Scheduling Info
1392. 8. I/O Status Info
1393. ```
1394.
1395. **Q3: Explain the Running State of a process.**
1396. ```
1397. Running State:
1398. - Process is using CPU
1399. - Instructions are being executed
1400. - PC continuously updates
1401. - Can transition to: Ready, Blocked, or Zombie
1402. ```
1403.
1404. **Q4: What happens when Context Switching is more frequent?**
1405. ```
1406. More Context Switching:
1407. - Higher CPU overhead
1408. - Lower system performance
1409. - Decreased CPU efficiency
1410. - Faster response time (benefit)
1411. ```
1412.
1413. **Q5: Explain the difference between Suspend and Terminate.**
1414. ```
1415. Suspend:
1416. - Temporary
1417. - Can be resumed
1418. - State preserved
1419.
1420. Terminate:
1421. - Permanent
1422. - Cannot be resumed
1423. - State destroyed
1424. ```

1425.
1426. **Q6: What are the causes of Context Switching?**
1427. ```
1428. 1. Time quantum expires
1429. 2. Higher priority process arrives
1430. 3. I/O request
1431. 4. Interrupt
1432. 5. System call
1433. 6. Process termination
1434. ```
1435.
1436. **Q7: Explain Suspended Ready state.**
1437. ```
1438. Suspended Ready:
1439. - Process was in Ready state
1440. - Suspended by OS
1441. - Not ready to execute immediately
1442. - Will move to Ready when resumed
1443. ```
1444.
1445. **Q8: What is the role of the Scheduler?**
1446. ```
1447. Scheduler:
1448. - Selects process from Ready Queue
1449. - Allocates CPU to selected process
1450. - Types: Long-term, Short-term, Medium-term
1451. - Determines which process runs next
1452. ```
1453.
1454. **Q9: Why does the OS suspend processes?**
1455. ```
1456. Reasons:
1457. 1. Memory Management (low RAM)
1458. 2. Power Saving (battery life)
1459. 3. Deadlock Recovery
1460. 4. Resource Management
1461. ```
1462.
1463. **Q10: What is the difference between Scheduler and Dispatcher?**
1464. ```
1465. Scheduler:
1466. - Decides which process to run
1467. - Makes selection decision
1468.

1469. Dispatcher:
1470. - Performs actual context switch
1471. - Loads and saves contexts
1472. ```
1473.
1474. ---
1475.
1476. ### 36. 5 Mark Questions with Answers
1477.
1478. **Q1: Explain the complete Process Life Cycle with all states and transitions.**
1479. ```
1480. PROCESS LIFE CYCLE:
1481.
1482. 1. NEW/EMBRYO State:
1483. - Process is being created
1484. - PID assigned
1485. - PCB created
1486. - Memory allocated
1487. - Transition: New → Ready (Admitted)
1488.
1489. 2. READY State:
1490. - Process is waiting for CPU
1491. - In Ready Queue
1492. - All resources allocated
1493. - Transition: Ready → Running (Scheduler dispatch)
1494.
1495. 3. RUNNING State:
1496. - Process using CPU
1497. - Instructions executing
1498. - Transition: Running → Ready (Time quantum expires)
1499. - Transition: Running → Blocked (I/O request)
1500. - Transition: Running → Zombie (Complete)
1501.
1502. 4. BLOCKED State:
1503. - Process waiting for I/O/Event
1504. - Not using CPU
1505. - Transition: Blocked → Ready (I/O complete)
1506.
1507. 5. ZOMBIE State:
1508. - Process completed
1509. - Only PCB remains
1510. - Waiting for parent to collect
1511. - Transition: Zombie → Destroyed

- 1512.
1513. Complete Transitions:
1514. New → Ready → Running → Ready (Time quantum)
1515. Running → Blocked → Ready → Running
1516. Running → Zombie → Destroyed
1517. ```
- 1518.
1519. **Q2: Explain Context Switching in detail with its steps, advantages, and disadvantages.**
1520. ```
1521. CONTEXT SWITCHING:
- 1522.
1523. Definition:
1524. The process of saving the context of one process and restoring the context of another.
- 1525.
1526. Steps of Context Switching:
- 1527.
1528. 1. SAVE CURRENT CONTEXT:
1529. - Save Program Counter to PCB
1530. - Save CPU Registers to PCB
1531. - Save Stack Pointer to PCB
1532. - Save Process State to PCB
- 1533.
1534. 2. UPDATE PCB:
1535. - Change state: Running → Ready/Blocked
1536. - Update scheduling info
- 1537.
1538. 3. SCHEDULER RUNS:
1539. - Select next process from Ready Queue
1540. - Based on scheduling algorithm
- 1541.
1542. 4. LOAD NEW CONTEXT:
1543. - Load PC from new PCB
1544. - Load Registers from new PCB
1545. - Load Stack Pointer from new PCB
- 1546.
1547. 5. RESUME EXECUTION:
1548. - Start executing new process
- 1549.
1550. ADVANTAGES:
1551. - Enables multitasking
1552. - Better CPU utilization
1553. - Fair scheduling

- 1554. - Improved responsiveness
- 1555. - Supports time-sharing
- 1556.
- 1557. DISADVANTAGES:
- 1558. - CPU overhead
- 1559. - Cache misses
- 1560. - TLB flush (in many systems)
- 1561. - Performance loss
- 1562. - Increased latency
- 1563. - Complex implementation
- 1564. ```
- 1565.
- 1566. **Q3: Explain Preemptive and Non-Preemptive Scheduling with advantages, disadvantages, and examples.**
- 1567. ```
- 1568. PREEMPTIVE SCHEDULING:
- 1569.
- 1570. Definition:
- 1571. OS can interrupt a running process and allocate CPU to another process.
- 1572.
- 1573. Features:
- 1574. - Process can be preempted
- 1575. - Higher priority processes get CPU quickly
- 1576. - Supports time-sharing
- 1577.
- 1578. Advantages:
- 1579. - Fast response time
- 1580. - Better CPU utilization
- 1581. - High priority processes execute quickly
- 1582. - Supports interactive systems
- 1583.
- 1584. Disadvantages:
- 1585. - More context switching
- 1586. - Higher CPU overhead
- 1587. - Complex implementation
- 1588.
- 1589. Examples:
- 1590. - Round Robin (RR)
- 1591. - Shortest Remaining Time First (SRTF)
- 1592. - Preemptive Priority Scheduling
- 1593.
- 1594. NON-PREEMPTIVE SCHEDULING:
- 1595.
- 1596. Definition:

1597. Process keeps CPU until it finishes or requests I/O.

1598.

1599. Features:

1600. - Process cannot be preempted

1601. - CPU released voluntarily

1602. - Simple implementation

1603.

1604. Advantages:

1605. - Less context switching

1606. - Lower CPU overhead

1607. - Simple implementation

1608. - Less CPU time wasted

1609.

1610. Disadvantages:

1611. - High priority processes may wait

1612. - Longer response time

1613. - Convoy effect possible

1614. - Process starvation possible

1615.

1616. Examples:

1617. - First Come First Served (FCFS)

1618. - Shortest Job First (SJF)

1619. - Non-Preemptive Priority Scheduling

1620.

1621. COMPARISON:

1622. | Feature | Preemptive | Non-Preemptive |

1623. |-----|-----|-----|

1624. | CPU Interruption | Yes | No |

1625. | Context Switching | More | Less |

1626. | Overhead | Higher | Lower |

1627. | Response Time | Faster | Slower |

1628. | Complexity | Higher | Lower |

1629. | Best For | Interactive | Batch |

1630. ```

1631.

1632. **Q4: Explain Process Control Block (PCB) in detail, including its components and importance.**

1633. ```

1634. PROCESS CONTROL BLOCK (PCB):

1635.

1636. Definition:

1637. PCB is a data structure maintained by the OS that contains all information about a process.

1638.

1639. PCB COMPONENTS:
1640.
1641. 1. PROCESS ID (PID):
1642. - Unique identifier for the process
1643.
1644. 2. PROCESS STATE:
1645. - Current state: New, Ready, Running, Blocked, Zombie
1646.
1647. 3. PROGRAM COUNTER (PC):
1648. - Address of next instruction to execute
1649.
1650. 4. CPU REGISTERS:
1651. - General purpose registers content
1652.
1653. 5. STACK POINTER (SP):
1654. - Current stack top address
1655.
1656. 6. MEMORY MANAGEMENT INFO:
1657. - Page table base
1658. - Memory limits
1659.
1660. 7. SCHEDULING INFORMATION:
1661. - Process priority
1662. - Time quantum used
1663.
1664. 8. I/O STATUS INFORMATION:
1665. - List of open files
1666. - I/O operations in progress
1667.
1668. 9. ACCOUNTING INFORMATION:
1669. - CPU time used
1670. - Process start time
1671.
1672. PCB LOCATION:
1673. RAM (Main Memory)
1674. └─ Kernel Space
1675. └─ Process Control Block (PCB)
1676.
1677. IMPORTANCE OF PCB:
1678.
1679. 1. Process Identification:
1680. - Contains unique PID
1681. - Identifies process to OS
1682.

1683. 2. Context Switching:
1684. - Stores process context
1685. - Enables switching between processes
- 1686.
1687. 3. Process Management:
1688. - Tracks process state
1689. - Controls process execution
- 1690.
1691. 4. Resource Management:
1692. - Tracks allocated resources
1693. - Manages memory and I/O
- 1694.
1695. 5. Accounting:
1696. - Tracks CPU usage
1697. - Monitors process behavior
- 1698.

1699. PCB STRUCTURE:

1700.			
1701.	PROCESS CONTROL BLOCK		
1702.			
1703.	PID: 1234		
1704.	State: Running		
1705.	PC: 0x004A3F2		
1706.	Registers: ...		
1707.	SP: 0x7FFFFFFF		
1708.	Memory Info: ...		
1709.	Priority: 5		
1710.	Open Files: ...		
1711.			

1712. ```

1713.

1714. **Q5: Explain Process Suspension in detail, including its causes, operations, and comparison with termination.**

1715. ```

1716. PROCESS SUSPENSION:

1717.

1718. Definition:

1719. Process Suspension is temporarily stopping a process without terminating it, while preserving its execution state.

1720.

1721. SUSPENSION OPERATIONS:

1722.

1723. A. STATE CHANGE:

1724. - Process state changes to:

1725. └─ Suspended Ready (if was in Ready)
 1726. └─ Suspended Blocked (if was in Blocked)
 1727.
 1728. B. RESOURCE RELEASE:
 1729. - Removed from CPU execution
 1730. - CPU usage becomes 0%
 1731. - CPU time becomes available
 1732.
 1733. C. CONTEXT SAVING:
 1734. - Save complete execution context:
 1735. └─ CPU Registers
 1736. └─ Program Counter (PC)
 1737. └─ Stack Pointer (SP)
 1738. └─ Stack
 1739. └─ Other processor state
 1740.
 1741. CAUSES OF SUSPENSION:
 1742.
 1743. 1. MEMORY MANAGEMENT:
 1744. - RAM is insufficient
 1745. - Less important processes suspended
 1746.
 1747. 2. POWER SAVING:
 1748. - Reduce power consumption
 1749. - Extend battery life
 1750.
 1751. 3. DEADLOCK RECOVERY:
 1752. - Resource conflicts
 1753. - System stability
 1754.
 1755. 4. USER REQUEST:
 1756. - Manual suspension
 1757.
 1758. SUSPEND vs TERMINATE:
 1759.
 1760. | Feature | Suspend | Terminate |
 1761. |-----|-----|-----|
 1762. | Duration | Temporary | Permanent |
 1763. | Resume | Yes | No |
 1764. | State | Preserved | Destroyed |
 1765. | CPU Usage | 0% | Released |
 1766. | Resources | Held | Released |
 1767. | PCB | Retained | Destroyed |
 1768.

1769. MEMORY MANAGEMENT:

1770.

1771. RAM (Main Memory)

1772. |— Kernel Space

1773. | |— PCB (Stays in RAM)

1774. | |— User Space

1775. | |— Process Memory

1776. | |

1777. | | (When low memory)

1778. ▼

1779. Secondary Storage

1780. |— Swap Space (Linux)

1781. |— Pagefile (Windows)

1782.

1783. SWAP OUT:

1784. - Process memory moved to disk

1785. - PCB remains in RAM

1786. - Process can be resumed later

1787.

1788. SLEEP MODE:

1789. - All processes suspended

1790. - State preserved in RAM

1791. - Computer enters low-power state

1792. - Resume when woken

1793.

1794. PROCESS STATE TRANSITION:

1795. Running → Suspending → Suspended Ready

1796. Running → Suspending → Suspended Blocked

1797. Suspended Ready → Ready (Resume)

1798. Suspended Blocked → Suspended Ready (I/O complete)

1799. ...

1800.

1801. ---

1802.

1803. ### 37. Viva Questions and Answers

1804.

1805. **Q1: What is a process?**

1806. > A process is a program in execution. It includes the program code, current activity, stack, data section, and process control block.

1807.

1808. **Q2: What is the difference between program and process?**

1809. > Program is a passive entity stored on disk, while process is an active entity loaded in memory and has execution state.

1810.

1811. **Q3: What is PCB and what does it contain?**

1812. > PCB (Process Control Block) is a data structure containing PID, process state, PC, registers, stack pointer, memory info, scheduling info, and I/O status.

1813.

1814. **Q4: Explain the various states of a process.**

1815. > New → Ready → Running → Blocked → Zombie. Process goes through these states during its lifetime.

1816.

1817. **Q5: What is context switching?**

1818. > Context switching is the process of saving the context of one process and loading the context of another.

1819.

1820. **Q6: What information is saved during context switching?**

1821. > Program Counter, CPU registers, Stack Pointer, and process state are saved.

1822.

1823. **Q7: Why is context switching needed?**

1824. > To enable multitasking, share CPU among processes, and provide fair scheduling.

1825.

1826. **Q8: What is the difference between preemptive and non-preemptive scheduling?**

1827. > Preemptive allows OS to interrupt processes, non-preemptive doesn't.

1828.

1829. **Q9: What is process suspension?**

1830. > Temporarily stopping a process without terminating it, preserving its state.

1831.

1832. **Q10: What is a zombie process?**

1833. > A process that has completed execution but its PCB remains until parent collects exit status.

1834.

1835. **Q11: What is the Ready Queue?**

1836. > A queue containing processes in Ready State waiting for CPU allocation.

1837.

1838. **Q12: What is swap space?**

1839. > Area on disk where OS moves process memory when RAM is insufficient.

1840.

1841. **Q13: What causes context switching?**

1842. > Time quantum expiry, higher priority process, I/O request, interrupt, system call, process termination.

1843.

1844. **Q14: What is dispatcher?**
1845. > Component that performs actual context switch and loads next process.
1846.
1847. **Q15: What is the difference between scheduler and dispatcher?**
1848. > Scheduler decides which process to run; dispatcher performs the actual switch.
1849.
1850. **Q16: What is Round Robin scheduling?**
1851. > Preemptive scheduling where each process gets a fixed time quantum in circular order.
1852.
1853. **Q17: What is priority scheduling?**
1854. > Scheduling based on process priority - higher priority processes get CPU first.
1855.
1856. **Q18: What is FCFS scheduling?**
1857. > Non-preemptive scheduling where processes are served in arrival order.
1858.
1859. **Q19: What is short-term scheduler?**
1860. > Selects process from Ready Queue for CPU allocation.
1861.
1862. **Q20: What is long-term scheduler?**
1863. > Selects processes to be loaded into memory.
1864.
1865. **Q21: What is medium-term scheduler?**
1866. > Removes/swaps processes from memory to disk.
1867.
1868. **Q22: What is a system call?**
1869. > Request from process to OS for service (I/O, file operations, etc.)
1870.
1871. **Q23: What is an interrupt?**
1872. > Signal from hardware or software requiring immediate attention.
1873.
1874. **Q24: What is I/O bound process?**
1875. > Process that spends more time on I/O operations than CPU.
1876.
1877. **Q25: What is CPU bound process?**
1878. > Process that spends more time on CPU operations than I/O.
1879.
1880. ---
1881.
1882. ### 38. MCQs for Practice
1883.
1884. **1. Which of the following is NOT a component of PCB?**

1885. a) PID
1886. b) Program Counter
1887. c) Process Memory (actual code)
1888. d) Process State
1889.
1890. **Answer: c) Process Memory (actual code)**
1891.
1892. **2. Context Switching is performed by:**
1893. a) Scheduler
1894. b) Dispatcher
1895. c) Both Scheduler and Dispatcher
1896. d) Neither
1897.
1898. **Answer: c) Both Scheduler and Dispatcher**
1899.
1900. **3. In which state does a process wait for I/O?**
1901. a) Ready
1902. b) Running
1903. c) Blocked
1904. d) Zombie
1905.
1906. **Answer: c) Blocked**
1907.
1908. **4. Preemptive Scheduling allows:**
1909. a) Process to run until completion
1910. b) OS to interrupt running process
1911. c) Process to request I/O
1912. d) Process to terminate
1913.
1914. **Answer: b) OS to interrupt running process**
1915.
1916. **5. Zombie state means:**
1917. a) Process is running
1918. b) Process is waiting for I/O
1919. c) Process completed but PCB remains
1920. d) Process is suspended
1921.
1922. **Answer: c) Process completed but PCB remains**
1923.
1924. **6. Process Suspension is:**
1925. a) Permanent termination
1926. b) Temporary stopping
1927. c) CPU allocation
1928. d) Memory allocation

1929.
1930. **Answer: b) Temporary stopping**
1931.
1932. **7. Which scheduler runs most frequently?**
1933. a) Long-term scheduler
1934. b) Short-term scheduler
1935. c) Medium-term scheduler
1936. d) All run equally
1937.
1938. **Answer: b) Short-term scheduler**
1939.
1940. **8. Context Switching overhead includes:**
1941. a) Saving registers
1942. b) Loading registers
1943. c) Cache effects
1944. d) All of the above
1945.
1946. **Answer: d) All of the above**
1947.
1948. **9. Which is an example of Preemptive Scheduling?**
1949. a) FCFS
1950. b) SJF
1951. c) Round Robin
1952. d) Non-Preemptive Priority
1953.
1954. **Answer: c) Round Robin**
1955.
1956. **10. PCB is stored in:**
1957. a) Secondary Storage
1958. b) Kernel Space in RAM
1959. c) User Space in RAM
1960. d) Cache Memory
1961.
1962. **Answer: b) Kernel Space in RAM**
1963.
1964. **11. Context switching happens when:**
1965. a) Time quantum expires
1966. b) Higher priority process arrives
1967. c) I/O request occurs
1968. d) All of the above
1969.
1970. **Answer: d) All of the above**
1971.
1972. **12. In Suspended Ready state:**

1973. a) Process is running
1974. b) Process is waiting for CPU
1975. c) Process is suspended and in memory/swap
1976. d) Process has completed
- 1977.
1978. **Answer: c) Process is suspended and in memory/swap**
- 1979.
1980. **13. Which is NOT an advantage of Preemptive Scheduling?**
1981. a) Fast response time
1982. b) Better CPU utilization
1983. c) Less context switching
1984. d) High priority process execution
- 1985.
1986. **Answer: c) Less context switching**
- 1987.
1988. **14. Which is an advantage of Non-Preemptive Scheduling?**
1989. a) Fast response time
1990. b) Lower CPU overhead
1991. c) Better CPU utilization
1992. d) High priority process execution
- 1993.
1994. **Answer: b) Lower CPU overhead**
- 1995.
1996. **15. PID stands for:**
1997. a) Process Information Data
1998. b) Process Identification Data
1999. c) Process ID
2000. d) Program ID
- 2001.
2002. **Answer: c) Process ID**
- 2003.
2004. ---
- 2005.
2006. ### 39. Important Definitions (Quick Revision)
- 2007.
2008. | Term | Definition |
2009. |-----|-----|
2010. | **Process** | A program in execution |
2011. | **PCB** | Data structure containing all process information |
2012. | **PID** | Unique identification number for process |
2013. | **Context Switching** | Saving state of one process and loading state of another |
2014. | **Preemptive Scheduling** | OS can interrupt and take CPU from process |

2015. | **Non-Preemptive Scheduling** | Process keeps CPU until done |

2016. | **Process Suspension** | Temporarily stopping a process |

2017. | **Zombie State** | Process completed but PCB remains |

2018. | **Ready State** | Process waiting for CPU |

2019. | **Running State** | Process using CPU |

2020. | **Blocked State** | Process waiting for I/O |

2021. | **Embryo State** | Process being created |

2022. | **Scheduler** | Selects process for CPU |

2023. | **Dispatcher** | Performs context switch |

2024. | **Swap Space** | Disk area for process memory |

2025.

2026. ---

2027.

2028. ### 40. Important Diagrams for Exams

2029.

2030. **Diagram 1: Process State Diagram**

2031. ```

2032. New → Ready → Running → Zombie

2033. Running → Ready (Time quantum)

2034. Running → Blocked (I/O)

2035. Blocked → Ready (I/O complete)

2036. ```

2037.

2038. **Diagram 2: Context Switching Flow**

2039. ```

2040. Process A Running

2041. ↓

2042. Save Context to PCB A

2043. ↓

2044. Select Next Process (Scheduler)

2045. ↓

2046. Load Context from PCB B

2047. ↓

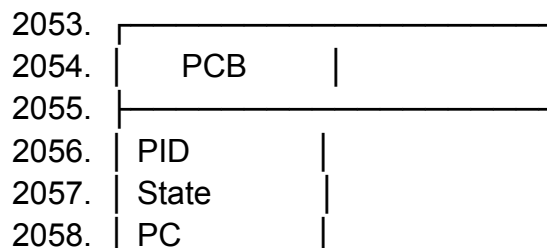
2048. Process B Running

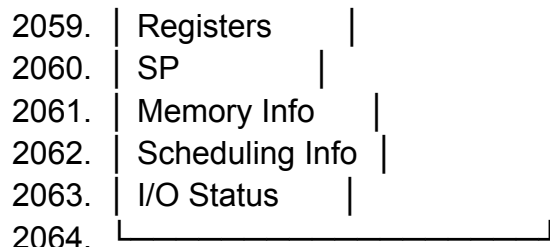
2049. ```

2050.

2051. **Diagram 3: PCB Structure**

2052. ```





2065. ```

2066.

2067. ****Diagram 4: Memory Layout****

2068. ```

2069. Stack (High Address)

2070. ↓

2071. (Free Space)

2072. ↑

2073. Heap

2074. Data Section

2075. Text Section

2076. (Low Address)

2077. ```

2078.

2079. ****Diagram 5: Scheduling Queues****

2080. ```

2081. Job Queue → Ready Queue → CPU

2082. ↓

2083. Device Queue

2084. ```

2085.

2086. ****Diagram 6: Suspension Transition****

2087. ```

2088. Running → Suspending → Suspended Ready

2089. Running → Suspending → Suspended Blocked

2090. ```

2091.

2092. ---

2093.

2094. **### 41. Short Notes for Revision**

2095.

2096. ****Process:****

2097. > A process is a program in execution. It includes code, data, stack, heap, and PCB.

2098.

2099. ****PCB:****

2100. > Data structure with all process info - PID, state, PC, registers, etc. Stored in kernel space.

2101.
2102. ****States:****
2103. > New → Ready → Running → Blocked → Zombie
2104.
2105. ****Context Switching:****
2106. > Saving one process context, loading another. Done by dispatcher.
2107.
2108. ****Preemptive Scheduling:****
2109. > OS can interrupt and take CPU. More overhead, better response.
2110.
2111. ****Non-Preemptive Scheduling:****
2112. > Process keeps CPU until done. Less overhead, simpler.
2113.
2114. ****Suspension:****
2115. > Temporary stop. State preserved. Can be resumed.
2116.
2117. ****Zombie:****
2118. > Complete but PCB remains. Parent collects exit status.
2119.
2120. ****Scheduler:****
2121. > Selects process for CPU. Types: Long, Short, Medium-term.
2122.
2123. ****Dispatcher:****
2124. > Performs actual context switch and context loading.
2125.
2126. ****PID:****
2127. > Unique ID for each process. Assigned when created.
2128.
2129. ****Swap Space:****
2130. > Disk area for moving process memory when RAM is low.
2131.
2132. ****Sleep Mode:****
2133. > All processes suspended. State in RAM. Low power.
2134.
2135. ****Context Switch Overhead:****
2136. > Time wasted on saving/loading contexts. Cache and TLB effects.
2137.
2138. ---
2139.
2140. ##  Complete Chapter Checklist
2141.
2142. - [x] Chapter 1: Introduction to Process
2143. - [x] Process
2144. - [x] Program vs Process

- 2145. - [x] PCB
- 2146. - [x] PID
- 2147.
- 2148. - [x] Chapter 2: Process States
- 2149. - [x] Embryo/New State
- 2150. - [x] Ready State
- 2151. - [x] Running State
- 2152. - [x] Blocked State
- 2153. - [x] Zombie State
- 2154. - [x] Complete State Diagram
- 2155.
- 2156. - [x] Chapter 3: Process Suspension
- 2157. - [x] Process Suspension
- 2158. - [x] Suspend vs Terminate
- 2159. - [x] Suspended Ready
- 2160. - [x] Suspended Blocked
- 2161. - [x] Swap Space
- 2162. - [x] Sleep Mode
- 2163.
- 2164. - [x] Chapter 4: Context Switching
- 2165. - [x] Context Switching Definition
- 2166. - [x] Information Saved
- 2167. - [x] Steps of Context Switching
- 2168. - [x] Overhead
- 2169. - [x] Advantages/Disadvantages
- 2170. - [x] More vs Less Context Switching
- 2171. - [x] Causes of Context Switching
- 2172.
- 2173. - [x] Chapter 5: Scheduling
- 2174. - [x] Preemptive Scheduling
- 2175. - [x] Non-Preemptive Scheduling
- 2176. - [x] Comparison
- 2177.
- 2178. - [x] Chapter 6: Scheduling Queues
- 2179. - [x] Job Queue
- 2180. - [x] Ready Queue
- 2181. - [x] Device Queue
- 2182.
- 2183. - [x] Chapter 7: Diagrams and Tables
- 2184. - [x] Complete Diagrams
- 2185. - [x] Comparison Tables
- 2186.
- 2187. - [x] Chapter 8: Exam Questions
- 2188. - [x] 1 Mark Questions

2189. - [x] 2 Mark Questions
2190. - [x] 5 Mark Questions
2191. - [x] Viva Questions
2192. - [x] MCQs
2193. - [x] Definitions
2194. - [x] Short Notes
2195.
2196. ---
2197.
2198. ** 📌 Important Points to Remember:**
2199.
2200. 1. **PCB** is in Kernel Space, not user space
2201. 2. **PID** is unique for each process
2202. 3. **Context Switching** saves only context, not program code
2203. 4. **Suspension** keeps PCB in RAM, may swap memory to disk
2204. 5. **Preemptive = OS can interrupt**, Non-Preemptive = Process keeps CPU
2205. 6. **Zombie = completed but not destroyed**
2206. 7. **Scheduler** decides, **Dispatcher** executes
2207. 8. **More context switching = More overhead, Better response**
2208. 9. **Less context switching = Less overhead, Worse response**
2209. 10. **Swap Space** is on disk, not in RAM
2210.
2211. ---
2212.
2213. **"Success is the sum of small efforts repeated day in and day out."**
2214.
2215. **Good Luck for Your Exams! 📚🎯**
2216.